DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 A

ALGORITHMIC DESIGN THROUGH PSEUDOCODE

TASK 1 – ALGORITHM DESIGN REVIEW

Diagnose and Repair Poor Pseudocode

Task 1.1 – Diagnose the Design

The proposed algorithm has the following deficiencies:

i) Incomplete Input Specification

The algorithm only states "Read packages" without specifying package weights, values and vehicle capacity. Without these inputs, the algorithm cannot determine which packages can be loaded.

ii) Undefined Selection Criterion

"Select the best package" does not explain what makes a package best. Since packages can be divided, the algorithm must compare their value-to-weight ratios.

iii) No Feasibility Check

The algorithm does not verify whether the selected package can fit within the remaining vehicle capacity. This may cause the vehicle capacity to be exceeded.

iv) No Partial Package Handling

The problem allows packages to be divided, but the algorithm does not explain how to load a fraction of a package when its complete weight cannot fit.

v) Remaining Capacity Is Not Updated

After loading a package, the remaining vehicle capacity must be reduced. Otherwise, the algorithm cannot correctly determine how much additional weight can be loaded.

vi) Undefined Termination Condition

"WHILE vehicle has space" does not specify what happens when no packages remain or when the remaining capacity becomes zero.

vii) Incomplete Result Construction

The algorithm does not specify how to calculate the total value obtained from complete and fractional packages.

Task 1.2 – Repair the Design

Algorithm: Fractional Knapsack Using Greedy Strategy

Objective: Maximise the total package value without exceeding vehicle capacity.

Greedy Choice: Select the package with the highest value-to-weight ratio first.

Pseudocode:

```
ALGORITHM FractionalKnapsack(P, n, W)
INPUT: n packages with weight and value, capacity W
OUTPUT: Maximum total value and selected fractions
BEGIN
  FOR each package i = 1 TO n DO
    Ratio[i] ← Value[i] / Weight[i]
  END FOR
  SORT packages by Ratio in descending order
  Remaining ← W
  TotalValue ← 0
  FOR i ← 1 TO n DO
    IF Remaining = 0 THEN
      BREAK
    END IF
```

```
        IF Weight[i] ≤ Remaining THEN
            Fraction[i] ← 1
            Remaining ← Remaining - Weight[i]
        ELSE
            Fraction[i] ← Remaining / Weight[i]
            Remaining ← 0
        END IF
        TotalValue ← TotalValue +
                Fraction[i] × Value[i]
    END FOR
    RETURN Fraction, TotalValue
END
```

Step-by-Step Explanation

1. Calculate each package's value-to-weight ratio and sort in descending order.
2. Initialise the remaining capacity and total value.
3. Select the highest-ratio package first.
4. Load it fully if it fits; otherwise, load only the fraction that fits.
5. Update the remaining capacity and total value.
6. Repeat until the capacity is full or all packages are considered.

Why is this a Greedy Algorithm?

It makes the locally optimal choice at every step by selecting the package that provides the maximum value per unit of weight.

Since fractional selection is permitted, taking the highest-ratio package first is optimal for the fractional knapsack problem.

Task 1.3 – Design Defence

The rewritten solution qualifies as pseudocode because it uses structured algorithmic statements such as FOR, IF, ELSE, BREAK and RETURN. It clearly specifies inputs, processing, decisions and outputs without depending on any particular programming language. It is more precise than informal English because every selection and feasibility condition is explicitly defined. It can be translated into different programming languages without changing its underlying logic.

TASK 2 – FROM BUSINESS REQUIREMENT TO GREEDY PSEUDOCODE

Job Sequencing with Deadlines

Task 2.1 – Model the Requirement

i) Input

A collection of n independent jobs, where each job contains:

- Job ID.
- Deadline.
- Profit.

Each job requires exactly one processing slot.

ii) Output

The final scheduled jobs, their assigned time slots and the maximum total profit.

iii) Objective

Maximise the total profit earned by completing jobs before or on their deadlines.

iv) Constraints

- Only one job can occupy a slot.
- Each job requires one slot.
- A job earns profit only if completed before its deadline.
- A job cannot be scheduled more than once.

v) Decision to be Made

Determine which job should be selected and in which available time slot it should be placed.

vi) Information Required

The algorithm checks each job's profit, deadline, available slots, and current schedule to decide whether it can be added.

Task 2.2 – Formulate a Greedy Hypothesis

Greedy Rule:

Select jobs in descending order of profit and assign each job to the latest available time slot on or before its deadline.

Justification:

Selecting higher-profit jobs first prioritises greater rewards. Placing each job in the latest feasible slot preserves earlier slots for jobs having tighter deadlines.

Task 2.3 – Design the Algorithm

Algorithm: Job Sequencing with Deadlines

```
ALGORITHM JobSequencing(Jobs, n)
INPUT: Jobs with ID, Deadline and Profit
OUTPUT: Final schedule and maximum profit
BEGIN
    SORT Jobs by Profit in descending order
    MaxD ← maximum deadline
    Initialise all slots from 1 TO MaxD as EMPTY
    TotalProfit ← 0
    FOR each job J in Jobs DO
        FOR t ← MIN(J.Deadline, MaxD) DOWNTO 1 DO
            IF Slot[t] = EMPTY THEN
```

```
            Slot[t] ← J.ID
            TotalProfit ← TotalProfit + J.Profit
            BREAK
        END IF
    END FOR
  END FOR
  RETURN Slot, TotalProfit
END
```

Step-by-Step Explanation

1. Sort all jobs according to descending profit.
2. Find the maximum deadline to determine the number of slots.
3. Initially mark every slot as empty.
4. Consider jobs from highest profit to lowest profit.
5. Search backwards from the job's deadline.
6. Place the job in the latest available slot.
7. If no feasible slot exists, reject that job.
8. Add the selected job's profit to the total.
9. Return the final schedule and total profit.

Task 2.4 – Desk-Check

Job	Deadline	Profit
A	2	80
B	1	35
C	2	60
D	3	40
E	1	25

Step 1: Sort by Descending Profit

A → C → D → B → E

Step 2: Maximum Deadline

Maximum deadline = 3

Available slots = 1, 2, 3.

Step 3: Trace the Decisions

Job	Profit	Decision	Reason
A	80	Slot 2	Latest available slot
C	60	Slot 1	Slot 2 occupied
D	40	Slot 3	Slot 3 available
B	35	Rejected	Slot 1 occupied
E	25	Rejected	No available slot

Final Schedule:

Slot	Selected Job
1	C
2	A
3	D

Total Profit:

= 60 + 80 + 40

= **180**

Therefore, maximum profit = **180**.

Task 2.5 – Challenge Your Design

Modified Input:

Change the profit of job B from 35 to 100.

Prediction:

The schedule will change because job B becomes the highest-profit job. It is expected to occupy slot 1, potentially displacing job C.

Modified Profit Order:

B → A → C → D → E

Relevant Decision Trace:

Job	Decision	Reason
B	Slot 1	Highest profit and deadline 1
A	Slot 2	Latest available slot
C	Rejected	Slots 1 and 2 occupied
D	Slot 3	Available
E	Rejected	Slot 1 occupied

Modified Schedule:

Slot	Job
1	B
2	A
3	D

New Total Profit:

= 100 + 80 + 40

= **220**

Conclusion:

The prediction was correct. Changing job B's profit changed its priority and displaced job C. The algorithm adapted to the modified input while maintaining the deadline constraints.

TASK 3 – FROM RECURRENCE TO EXECUTABLE LOGIC

Dynamic Programming – 0/1 Knapsack

Task 3.1 – Define the State

Let:

DP[i][w] represent the maximum total value obtainable by selecting items from the first i items without exceeding a capacity of w.

Here:

- i represents the number of items considered.
- w represents the available capacity.
- DP[i][w] stores the optimal value for that subproblem.

Task 3.2 – Develop the Recurrence

For every item i, there are two possible decisions:

i) Exclude the Current Item

If the current item is not selected, the maximum value remains:

DP[i-1][w]

This means the solution is obtained using the previous items only.

ii) Include the Current Item

If the item's weight is less than or equal to the available capacity, it can be selected.

The resulting value is:

Value[i] + DP[i-1][w - Weight[i]]

Here, the item's value is added to the optimal value obtained using the remaining capacity.

iii) Recurrence Relation

If Weight[i] ≤ w:

DP[i][w] = MAX(

DP[i-1][w],

Value[i] + DP[i-1][w-Weight[i]]

)

Otherwise:

DP[i][w] = DP[i-1][w]

iv) Boundary Conditions

DP[0][w] = 0

DP[i][0] = 0

These conditions indicate that zero items or zero capacity cannot produce any positive value.

Task 3.3 – Translate the Recurrence into Pseudocode

Algorithm: 0/1 Knapsack Using Dynamic Programming

ALGORITHM Knapsack(Weight, Value, n, W)

INPUT: n items and capacity W

OUTPUT: Maximum value and selected items

```
BEGIN
   CREATE DP[0..n][0..W]
   FOR w ← 0 TO W DO
      DP[0][w] ← 0
   END FOR
   FOR i ← 0 TO n DO
      DP[i][0] ← 0
   END FOR
   FOR i ← 1 TO n DO
      FOR w ← 1 TO W DO
         IF Weight[i] ≤ w THEN
            DP[i][w] ← MAX(
```

```
            DP[i-1][w],
            Value[i] + DP[i-1][w-Weight[i]]
        )
      ELSE
        DP[i][w] ← DP[i-1][w]
      END IF
    END FOR
  END FOR
  MaxValue ← DP[n][W]
  w ← W
  FOR i ← n DOWNTO 1 DO
    IF DP[i][w] ≠ DP[i-1][w] THEN
      SELECT Item[i]
      w ← w - Weight[i]
    END IF
  END FOR
  RETURN MaxValue, SelectedItems
END
```

Step-by-Step Explanation

1. Create a two-dimensional DP table.
2. Initialise the first row and first column to zero.
3. Process items one by one.
4. For each capacity, check whether the current item fits.
5. If it fits, compare inclusion and exclusion values.

6. Store the maximum value.
7. After completing the table, obtain the optimum value from DP[n][W].
8. Trace backwards through the table to identify the selected items.
9. Reduce the remaining capacity whenever an item is selected.
10. Return both the maximum value and selected items.

Why is Reconstruction Necessary?

The final DP value only provides the maximum achievable value. Backtracking identifies the actual items responsible for producing that value.

Task 3.4 – Desk-Check

Given Input:

Item	Weight	Value
P	2	12
Q	3	20
R	4	25
S	5	30

Vehicle capacity = 7.

Important DP States:

Items Considered	Capacity 7	Selected Combination
P	12	P
P, Q	32	P + Q
P, Q, R	37	P + R
P, Q, R, S	45	Q + R

Important Calculations:

For item R:

DP[3][7] = MAX(DP[2][7], 25 + DP[2][3])

= MAX(32, 25 + 12)

= 37

For item S:

DP[4][7] = MAX(DP[3][7], 30 + DP[3][2])

= MAX(37, 30 + 12)

= 42

Correction: the inclusion value for S is 42, so:

DP[4][7] = **42**, not 45.

The optimal combination is P + S.

Final Selection:

P: Weight 2, Value 12.

S: Weight 5, Value 30.

Total weight = 2 + 5 = 7.

Maximum value = 12 + 30 = **42**.

Task 3.5 – Explain the Dependency

Dynamic Programming states depend on previously calculated smaller subproblems. Therefore, the required previous states must be available before calculating a new state.

In this algorithm, DP[i][w] depends on DP[i-1][w] and DP[i-1][w-Weight[i]]. Hence, items are processed in increasing order and capacities are evaluated using the previous row.

This dependency order prevents the use of uncomputed values and ensures that every state is calculated correctly.

TASK 4 – PSEUDOCODE FORENSICS

Find, Explain and Repair an Algorithmic Defect

Task 4.1 – Locate the Defect

The exact defect is in the following condition:

IF substring(Text, j, i) is in Dictionary

The algorithm checks only whether the substring exists in the dictionary. It does not verify whether the prefix before that substring can also be segmented.

The correct decision must consider two conditions:

1. The previous prefix DP[j] must be TRUE.
2. The substring from j to i must exist in the dictionary.

Without checking DP[j], an invalid prefix may incorrectly be considered segmentable.

Task 4.2 – Construct a Failing Input

i) Text

Text = "ab"

ii) Dictionary

Dictionary = {"b"}

iii) Expected Result

FALSE, because the character "a" is not present in the dictionary.

iv) Faulty Algorithm Trace

Initial state:

DP[0] = TRUE

i	j	Substring	In Dictionary?	DP[i]
1	0	a	No	FALSE
2	0	ab	No	FALSE
2	1	b	Yes	TRUE

At i = 2, the algorithm finds "b" in the dictionary and marks DP[2] as TRUE.

However, DP[1] is FALSE, meaning the preceding prefix "a" cannot be segmented.

Therefore, DP[2] becomes incorrectly TRUE.

First Incorrect State:

DP[2] = TRUE.

Expected DP[2] = FALSE.

Task 4.3 – Repair the Algorithm

Only the defective condition needs to be modified.

Corrected Condition:

```
IF DP[j] = TRUE AND
  substring(Text, j, i) is in Dictionary THEN
   DP[i] ← TRUE
   BREAK
END IF
```

Why This Correction Works

The corrected condition verifies both the previous prefix and the current dictionary word. Therefore, a complete segmentation is accepted only when every part of the string can be formed using valid dictionary words.

Task 4.4 – Regression Check

Input:

Text = "cat"

Dictionary = {"cat"}

Trace:

DP[0] = TRUE.

For i = 1, substring "c" is not found.

For i = 2, substrings "c" and "ca" are not found.

For i = 3, substring "cat" is found.

Since DP[0] = TRUE, the corrected condition succeeds.

Therefore:

DP[3] = TRUE.

Comparison:

Algorithm	Result
Original algorithm	TRUE
Corrected algorithm	TRUE
Expected result	TRUE

Conclusion:

The correction eliminates the incorrect acceptance of invalid prefixes while preserving the original algorithm's correct behaviour for valid segmentations.

TASK 5 – SAME GRAPH, DIFFERENT ALGORITHMIC REASONING

Dijkstra, Bellman-Ford and Floyd

Given Directed Weighted Graph

Edge	Weight
A → B	4
A → C	7
B → C	2
B → D	6
C → D	1
D → E	3
C → E	8

Task 5.1 – Greedy Shortest-Path Design

Algorithm: Dijkstra's Shortest Path

Objective: Find the shortest distance from source A to every other vertex.

Pseudocode:

```
ALGORITHM Dijkstra(G, A)
INPUT: Directed graph G with non-negative weights
OUTPUT: Shortest distances and predecessors
BEGIN
   FOR each vertex v DO
      Dist[v] ← ∞
      Pred[v] ← NIL
      Visited[v] ← FALSE
   END FOR
   Dist[A] ← 0
   REPEAT |V| TIMES
      u ← unvisited vertex with minimum Dist[u]
      IF Dist[u] = ∞ THEN
         BREAK
      END IF
      Visited[u] ← TRUE
      FOR each neighbour v of u DO
         IF Visited[v] = FALSE AND
            Dist[u] + Weight(u,v) < Dist[v] THEN
            Dist[v] ← Dist[u] + Weight(u,v)
```

```
            Pred[v] ← u
        END IF
    END FOR
  END REPEAT
  RETURN Dist, Pred
END
```

Step-by-Step Explanation

1. Initialise all distances to infinity.
2. Set the source distance to zero.
3. Select the unvisited vertex with minimum distance.
4. Mark it as visited.
5. Examine all its outgoing edges.
6. Calculate the alternative distance through the selected vertex.
7. Update the distance and predecessor if a shorter path is found.
8. Repeat until all reachable vertices are processed.

Relaxation Formula:

Dist[v] = MIN(Dist[v], Dist[u] + Weight(u,v))

Desk-Check : Initial Distances:

A = 0, B = ∞, C = ∞, D = ∞, E = ∞.

Selected Vertex	B	C	D	E
A	4	7	∞	∞
B	4	6	10	∞
C	4	6	7	14
D	4	6	7	10
E	4	6	7	10

Final Shortest Distances:

Vertex	Shortest Distance	Shortest Path
A	0	A
B	4	A → B
C	6	A → B → C
D	7	A → B → C → D
E	10	A → B → C → D → E

Conclusion:

The shortest distance from A to E is 10.

Task 5.2 – Change the Graph Assumption

Dijkstra's algorithm assumes all edge weights are non-negative.

i) Unsafe Greedy Decision

The selection of the unvisited vertex with the smallest temporary distance is treated as final.

If negative-weight edges exist, a later path may reduce the distance of an already finalised vertex. Therefore, the greedy commitment becomes unsafe.

ii) Bellman-Ford Pseudocode

```
ALGORITHM BellmanFord(G, A)

INPUT: Graph G and source A

OUTPUT: Shortest distances or negative cycle

BEGIN

  FOR each vertex v DO

    Dist[v] ← ∞

    Pred[v] ← NIL

  END FOR

  Dist[A] ← 0
```

```
    FOR i ← 1 TO |V|-1 DO
        FOR each edge (u,v) DO
            IF Dist[u] ≠ ∞ AND
                Dist[u] + Weight(u,v) < Dist[v] THEN
                Dist[v] ← Dist[u] + Weight(u,v)
                Pred[v] ← u
            END IF
        END FOR
    END FOR
    FOR each edge (u,v) DO
        IF Dist[u] ≠ ∞ AND
            Dist[u] + Weight(u,v) < Dist[v] THEN
            RETURN "Reachable Negative Cycle"
        END IF
    END FOR
    RETURN Dist, Pred
END
```

iii) Why Repeated Relaxation Is Required

A shortest path in a graph without negative cycles contains at most |V|-1 edges.

Each complete relaxation pass allows shortest-path information to propagate through additional edges. Therefore, |V|-1 passes are sufficient to determine the shortest distances.

iv) Negative Cycle Detection

After |V|-1 passes, one additional pass is performed.

If any reachable edge can still reduce a distance, a reachable negative-weight cycle exists.

Task 5.3 – Change the Requirement

i) State Definition

$D^{(k)}[i][j]$ represents the shortest distance from vertex i to vertex j using only the first k vertices as possible intermediate vertices.

ii) Floyd's State Transition

$D^{(k)}[i][j] = MIN($
$D^{(k-1)}[i][j],$
$D^{(k-1)}[i][k] + D^{(k-1)}[k][j]$
$)$

iii) Pseudocode

```
ALGORITHM FloydWarshall(D, n)
INPUT: Distance matrix D
OUTPUT: All-pairs shortest distances
BEGIN
   FOR k ← 1 TO n DO
      FOR i ← 1 TO n DO
         FOR j ← 1 TO n DO
            D[i][j] ← MIN(
               D[i][j],
               D[i][k] + D[k][j]
            )
         END FOR
      END FOR
   END FOR
   RETURN D
END
```

iv) Explanation

For every pair of vertices, the algorithm checks whether travelling through an intermediate vertex k provides a shorter path. If it does, the distance is updated.

Difference Between the Approaches

Algorithm	Approach	Requirement
Dijkstra	Greedy	Single-source shortest paths with non-negative edges
Bellman-Ford	Repeated relaxation	Single-source shortest paths with negative edges
Floyd-Warshall	Dynamic Programming	All-pairs shortest paths

Dijkstra and Bellman-Ford calculate shortest paths from one source vertex. Floyd-Warshall calculates shortest distances between every pair of vertices using intermediate-vertex states.

TASK 6 – DESIGN CHALLENGE

Energy-Aware Job Scheduling in a Cloud Data Centre

Stage 1 – Interpret the Problem

i) Input

A collection of jobs containing:

- Job ID.
- Start time.
- Finish time.
- Energy-saving value.

ii) Output

A set of selected non-overlapping jobs and their maximum total energy-saving value.

iii) Objective

Maximise the total energy-saving value by selecting compatible jobs.

iv) Constraints

1. Only one job can execute at a time.
2. Two jobs with overlapping intervals cannot both be selected.
3. A job's start time must be earlier than its finish time.
4. If one job finishes exactly when another starts, both can be selected.
5. Each job can be selected at most once.

v) Decisions to Be Made

For every job, decide whether to include it or exclude it from the optimal schedule. The algorithm must consider the value of the current job together with the best compatible earlier schedule.

Stage 2 – Propose the Strategy

i) Possible Strategies

Strategy	Decision Principle	Limitation
Earliest Finish Time	Select the earliest finishing job	Maximises job count, not energy value
Highest Value	Select the highest-value job	May block several valuable compatible jobs
Shortest Duration	Select the shortest job	May ignore high-value combinations
Dynamic Programming	Compare compatible combinations	Requires additional memory and computation

ii) Selected Strategy: Weighted Interval Scheduling Using Dynamic Programming

I select Dynamic Programming because the problem requires maximising total energy-saving value rather than simply selecting the maximum number of jobs.

A locally attractive job may overlap with several other jobs whose combined energy-saving value is greater.

Dynamic Programming considers both including and excluding each job and retains the optimal result of smaller subproblems.

iii) Why Not Use a Simple Greedy Strategy?

Consider a job with energy value 100 that occupies a long interval. Selecting it immediately may prevent the selection of three shorter, non-overlapping jobs with a combined value of 180.

Therefore, selecting the highest-value job first does not guarantee the global optimum.

Stage 3 – Identify the Critical Information

i) Preprocessing

Sort all jobs in ascending order of finish time.

This allows the algorithm to identify compatible previous jobs efficiently.

ii) State Definition

Let:

DP[i] represent the maximum total energy-saving value obtainable by selecting jobs from the first i jobs in sorted order.

iii) Compatibility Information

Let p(i) represent the index of the latest job before job i whose finish time is less than or equal to the start time of job i.

Mathematically:

$p(i) = \max \{j < i : Finish[j] \leq Start[i]\}$

If no compatible job exists, $p(i) = 0$.

iv) Decision Rule

For each job, two decisions are possible:

Include the job:

Energy[i] + DP[p(i)]

Exclude the job:

DP[i-1]

v) Recurrence Relation

DP[i] = MAX(
Energy[i] + DP[p(i)],
DP[i-1]
)

vi) Base Condition

DP[0] = 0

No jobs produce zero energy savings.

vii) Why This State Is Sufficient

The state stores the maximum energy-saving value from the first i jobs. The compatibility index identifies the best earlier subproblem that can be combined with the current job.

Thus, smaller optimal solutions can be combined to construct a larger optimal solution.

Stage 4 – Design Complete Pseudocode

Algorithm: Energy-Aware Job Scheduling Using Dynamic Programming

ALGORITHM EnergyScheduling(Jobs, n)

INPUT: Jobs with Start, Finish and Energy

OUTPUT: Selected jobs and maximum energy savings

BEGIN

SORT Jobs by Finish time in ascending order

```
FOR i ← 1 TO n DO
    p[i] ← 0
    FOR j ← 1 TO i-1 DO
        IF Finish[j] ≤ Start[i] THEN
            p[i] ← j
        END IF
    END FOR
END FOR
DP[0] ← 0
FOR i ← 1 TO n DO
    Include ← Energy[i] + DP[p[i]]
    Exclude ← DP[i-1]
    DP[i] ← MAX(Include, Exclude)
END FOR
Selected ← EMPTY
i ← n
WHILE i > 0 DO
    IF Energy[i] + DP[p[i]] > DP[i-1] THEN
        ADD Jobs[i] TO Selected
        i ← p[i]
    ELSE
        i ← i - 1
    END IF
END WHILE
```

```
    REVERSE Selected
    RETURN Selected, DP[n]
END
```

Step-by-Step Explanation

1. Sorting

Jobs are arranged in ascending order of finish time. This helps identify compatible earlier jobs.

2. Compatibility Calculation

For each job, the algorithm finds the latest previous job that finishes before or exactly when the current job starts.

3. Initialisation

DP[0] is assigned zero because no jobs can produce no energy savings.

4. Include Decision

The algorithm adds the current job's energy value to the optimal value of its compatible previous jobs.

5. Exclude Decision

The algorithm retains the optimal value obtained without selecting the current job.

6. State Update

The maximum of the two decisions is stored in DP[i].

7. Reconstruction

The algorithm traces backwards through the DP table to identify the actual selected jobs.

8. Final Output

The selected jobs and DP[n] are returned.

Stage 5 – Desk-Check

Given Dataset

Job	Start	Finish	Energy
J1	1	3	20
J2	2	5	50
J3	4	6	30
J4	6	8	40
J5	5	9	60
J6	8	10	30
J7	9	11	50
J8	11	13	40

Step 1: Sorting

The given jobs are already arranged in ascending order of finish time.

Step 2: Compatibility Calculation

Job	Finish Time	Compatible Previous Job p(i)
J1	3	0
J2	5	0
J3	6	J1
J4	8	J3
J5	9	J2
J6	10	J4
J7	11	J5
J8	13	J7

For example, J5 starts at time 5. J2 finishes at time 5, so J2 and J5 are compatible.

Step 3: DP Calculation

DP[i] = MAX(Energy[i] + DP[p(i)], DP[i-1])

Job	Include Value	Exclude Value	DP[i]
J1	20	0	20
J2	50	20	50
J3	50	50	50
J4	90	50	90
J5	110	90	110
J6	120	110	120
J7	160	120	160
J8	200	160	200

Important State Calculations

For J5:

DP[5] = MAX(60 + DP[2], DP[4])

= MAX(60 + 50, 90) = 110.

For J8:

DP[8] = MAX(40 + DP[7], DP[7])

= MAX(40 + 160, 160)

= 200.

Step 4: Reconstruction

Starting from DP[8]:

- J8 is selected because 200 > 160.
- Move to p(8) = 7.
- J7 is selected because 160 > 120.
- Move to p(7) = 5.
- J5 is selected because 110 > 90.

- Move to p(5) = 2.
- J2 is selected because 50 > 20.

Final Selected Jobs:

J2 → J5 → J7 → J8.

Compatibility Verification

J2: 2–5

J5: 5–9

J7: 9–11

J8: 11–13

All jobs are compatible because each preceding job finishes exactly when the next job starts.

Total Energy-Saving Value:

= 50 + 60 + 50 + 40

= **200 units**

Therefore, the maximum energy-saving value is **200 units**.

Stage 6 – Attack Your Own Design

i) Adversarial Test Design

To challenge the algorithm, consider a situation where one job has a very high energy-saving value but occupies a long interval, while several smaller jobs can collectively provide a greater benefit.

Constructed Dataset

Job	Start	Finish	Energy
A	1	10	100
B	1	4	60
C	4	7	60
D	7	10	60
E	10	12	20

ii) Why Is This Input Challenging?

Job A has the highest individual energy-saving value of 100. However, selecting A prevents selecting B, C and D together.

The combined energy-saving value of B, C and D is 180, which is greater than A's value.

iii) Prediction

A highest-value-first greedy algorithm would select A and then E.

Its total value would be:

100 + 20 = 120.

However, the Dynamic Programming algorithm should identify B, C, D and E as the better combination.

iv) Applying Dynamic Programming

All jobs are already sorted by finish time.

Job	p(i)	Include Value	Exclude Value	DP[i]
A	0	100	0	100
B	0	60	100	100
C	B	120	100	120
D	C	180	120	180
E	D	200	180	200

v) Reconstruction

- Job E is selected because 200 > 180.
- Job D is selected because 180 > 120.
- Job C is selected because 120 > 100.
- Job B is selected because 100 = 100?

At this point, reconstruction reaches the comparison for B. Since both inclusion and exclusion give 100, the pseudocode's strict greater-than condition excludes B. This would produce D and E, with value 80, which is inconsistent with the DP value.

Required Reconstruction Refinement

When both choices have equal values, either choice can produce an optimal solution.

For this dataset, use the condition:

IF Energy[i] + DP[p[i]] ≥ DP[i-1] THEN

ADD Jobs[i] TO Selected

i ← p[i]

ELSE

i ← i - 1

END IF

With this tie-breaking rule:

B, C, D and E are selected.

vi) Final Calculation

= 60 + 60 + 60 + 20 = **200 units**

vii) Conclusion

The adversarial input demonstrates that a locally attractive greedy decision may produce a poor overall result. Dynamic Programming evaluates both inclusion and exclusion decisions and preserves optimal subproblem results. Therefore, it successfully identifies the globally optimal schedule instead of relying on an immediate local choice.

OVERALL CONCLUSION

This assignment demonstrates the application of Greedy Algorithms, Dynamic Programming and shortest-path algorithms to different optimisation problems. Through problem modelling, state formulation, pseudocode design, manual tracing and adversarial testing, I understood that an algorithm must be justified by its decision-making logic rather than only its final output. I also learned that Greedy strategies are suitable when local decisions can guarantee a global optimum, whereas Dynamic Programming is useful when the optimal solution depends on combinations of smaller subproblems.